TeleDrive: edge-based two-wheeler riding behaviour analysis using smartphone inertial sensors and on-device machine learning

Sharan S [1], Prof. Sachin M [2]

[1] Institute of Computer Science and Information Sciences, Srinivas University, Mangaluru, Karnataka, India. ORCID-ID: [placeholder]. E-mail: [placeholder]

[2] Institute of Computer Science and Information Sciences, Srinivas University, Mangaluru, Karnataka, India. ORCID-ID: [placeholder]. E-mail: [placeholder]

ABSTRACT

Two-wheeler riders operating under time-constrained delivery conditions frequently develop aggressive riding habits that increase fuel consumption, mechanical wear, and accident probability. Affordable, hardware-free tools for monitoring such behaviour in real time are not widely available to individual riders. This paper describes TeleDrive, a standalone Android application that performs on-device riding behaviour analysis using only the smartphone's built-in accelerometer, gyroscope, GPS receiver, and rear camera. The system acquires linear acceleration and gyroscope data at approximately 50 Hz through an Android Foreground Service, accumulates readings into one-second sliding windows, and applies a three-stage signal-conditioning pipeline comprising spike rejection, median filtering, and moving-average smoothing. Eight statistical features are extracted per window and fed into one of three selectable detection modes: a speed-adaptive rule-based engine, an on-device one-dimensional Convolutional Neural Network deployed via TensorFlow Lite, or a Hybrid consensus of both engines running in parallel. The CNN model (version 4) was trained on 115,250 class-balanced samples derived from 473,800 raw sensor readings collected across 48 real two-wheeler ride sessions. Offline evaluation on a stratified hold-out set of 23,050 samples achieved 96.0% accuracy and a macro-average F1-score of 0.929. Real-world validation on 402 labelled windows from live rides achieved 98.76% accuracy. Confirmed events are scored using a diminishing-penalty eco score model, fuel wastage is estimated from mileage degradation per event type, and photographic evidence is automatically captured through CameraX. Trip data, route polylines, event markers, and multi-trip hotspot detection are visualised through an interactive Event Map. All sensing, inference, scoring, and storage occur entirely on-device without cloud connectivity.

Keywords: mobile telematics, riding behaviour analysis, 1D-CNN, TensorFlow Lite, edge AI, two-wheeler safety

1. INTRODUCTION:

Road safety among two-wheeler users constitutes a persistent public health and transportation challenge across developing economies. The World Health Organization estimates approximately 1.19 million road traffic deaths per year globally, with motorcyclists and other vulnerable road users accounting for more than half of that figure (WHO, 2023). In India, the Ministry of Road Transport and Highways recorded 4,61,312 road accidents and 1,68,491 fatalities in 2022, and analysis of official data consistently identifies two-wheeler users as the single largest affected group, responsible for nearly half of all road accident deaths in the country (MORTH, 2022).

The growth of platform-based delivery services — operating under strict per-order time windows — has added a behavioural dimension to this problem. Riders employed by hyperlocal logistics operators navigate dense urban traffic under constant deadline pressure, creating incentives for aggressive acceleration, late braking, and gap-taking manoeuvres. Research on powered two-wheelers shows that time pressure is directly associated with increased overspeeding, delayed hazard response, and braking inconsistency (Vlakveld et al., 2019). For a typical per-order earning of ₹15–25, the hidden costs of such behaviour — excess fuel consumption, brake wear, tyre degradation, and elevated accident probability — represent a financially and physically significant burden that most individual riders cannot quantify.

Commercial telematics platforms that could address this problem are largely inaccessible to individual riders. Systems such as Vamosys, Letstrack, and Fleetx are built for fleet operators, require OBD-II or GPS tracker hardware, and charge recurring subscription fees that are economically impractical for gig workers, commuters, and students. Smartphone-based alternatives exist, but the majority are calibrated for four-wheelers and apply detection thresholds and vibration models that do not transfer accurately to the lighter mass, different suspension dynamics, and distinct road-contact characteristics of two-wheelers. Cloud-dependent architectures further introduce latency, data costs, and continuous location tracking that are undesirable in privacy-sensitive individual use cases.

TeleDrive was developed to address these gaps directly. The system performs complete riding behaviour analysis on-device, using only the sensors already present in any mid-range Android smartphone — accelerometer, gyroscope, GPS, and camera. It requires no external hardware, no cloud backend, and no internet connectivity for its core detection, scoring, and storage pipeline. The application targets Android 8.0 (API 26) and later, covers approximately 95% of active Android devices, and is deployable at zero hardware cost for the rider. The system classifies three categories of unsafe riding behaviour — harsh acceleration, harsh braking, and unstable riding — and produces per-trip eco scores, fuel and financial impact estimates, evidential photographs, and a spatial Event Map with hotspot detection across multiple trips. The design, implementation, evaluation, and key engineering decisions underlying TeleDrive are described in the sections that follow.

2. OBJECTIVES:

(1) To continuously acquire accelerometer and gyroscope data at approximately 50 Hz during a trip through an Android Foreground Service, alongside GPS speed and location at one-second intervals.

(2) To apply a three-stage signal pre-processing pipeline and extract an eight-feature vector from each one-second sensor window.

(3) To detect harsh acceleration, harsh braking, and unstable riding events using a speed-adaptive rule-based engine with multi-layer validation logic.

(4) To classify riding windows using an on-device TensorFlow Lite 1D-CNN model, with hybrid consensus support when both engines operate in parallel.

(5) To reduce false positives through an event-confirmation state machine with consecutive detection entry thresholds and hysteresis-based exit logic.

(6) To compute a per-trip eco score between 0 and 100 using a diminishing-returns penalty model sensitive to event type and severity.

(7) To estimate per-trip fuel consumption and financial loss in Indian Rupees attributable to aggressive riding behaviour.

(8) To capture automatic photographic evidence through CameraX for confirmed non-normal events, with scheduled cleanup via WorkManager.

(9) To generate contextual, trend-based, and streak-based riding insights from accumulated trip history.

(10) To render an interactive Event Map with route polylines, colour-coded event markers, evidence matching, clustering, and multi-trip hotspot detection.

3. REVIEW OF LITERATURE/RELATED WORKS:

Smartphone-based driving behaviour analysis has attracted sustained research attention over the past decade, driven by the widespread availability of inertial measurement units in consumer mobile devices. The feasibility of using accelerometers and gyroscopes for detecting aggressive driving manoeuvres was established early, with Johnson and Trivedi (2011) demonstrating that a smartphone mounted in a vehicle could classify acceleration and braking events with adequate precision when appropriate signal conditioning was applied. That work, however, focused on four-wheelers and did not address the distinct vibration profile and motion dynamics of two-wheelers.

Subsequent studies applied machine learning to improve event classification accuracy. Deep learning approaches, particularly recurrent and convolutional networks applied to raw time-series sensor data, demonstrated advantages over feature-engineering pipelines for handling overlapping and subtle driving patterns (LeCun et al., 2015). One-dimensional CNNs, in particular, became a practical choice for mobile deployment because of their low parameter

counts relative to recurrent architectures, enabling real-time inference on constrained embedded processors (Ignatov, 2018).

A parallel strand of research examined the deployment constraints of on-device inference. TensorFlow Lite established a widely used framework for converting trained models into formats executable on Android and iOS devices within milliseconds per inference, while maintaining practical accuracy (Abebe et al., 2021). However, most deployment studies in this area continued to target car-based telematics scenarios.

The specific problem of two-wheeler riding behaviour detection has received comparatively less systematic study. The majority of smartphone telematics applications reviewed in the literature — including open platforms and published prototypes — were designed for cars. Two-wheeler riders face different challenges: higher sensitivity to road surface roughness, narrower wheelbase dynamics, and significantly smaller vehicle mass all alter the sensor signal characteristics of acceleration, braking, and instability events. Handel et al. (2014) identified this calibration gap in the context of insurance telematics and noted that smartphone-only solutions required careful context-specific tuning to avoid poor generalisation.

The table below summarises key prior works and their relationship to the problem addressed by TeleDrive.

Table 1: Summary of related works in smartphone-based driving behaviour analysis

Sl.No	Area & Focus of the Research	Result of the Research	Reference
1	Smartphone accelerometer for four-wheeler driving event detection	Acceleration and braking events classifiable using threshold logic with GPS speed validation	Johnson & Trivedi (2011)
2	Deep learning for time-series classification on mobile sensors	1D-CNN outperforms feature-based classifiers for motion event detection on constrained hardware	LeCun et al. (2015); Ignatov (2018)
3	TensorFlow Lite for on-device model inference	Sub-15 ms per-inference latency feasible on mid-range Android devices with acceptable accuracy	Abebe et al. (2021)
4	Insurance telematics calibration for smartphone-only platforms	Calibration gap exists between four-wheeler and two-wheeler contexts; context-specific tuning required	Handel et al. (2014)
5	Road safety among two-wheeler users in India	Two-wheelers account for nearly half of annual road accident deaths; aggressive riding worsens risk	MORTH (2022); WHO (2023)

TeleDrive differentiates itself from the prior landscape in three primary ways. First, the detection pipeline is designed and calibrated exclusively for two-wheelers rather than adapted from car-based systems. Second, the system integrates a hybrid detection architecture that combines deterministic rule logic with learned CNN inference on the same one-second window, with a physics-priority consensus protocol for disagreements. Third, the complete pipeline — sensor acquisition, signal conditioning, classification, scoring, evidence

handling, and storage — operates without any cloud component, addressing the privacy, latency, and connectivity concerns identified in prior work.

4. MATERIALS AND METHODS:

4.1 System architecture and technology stack

TeleDrive is developed in Kotlin (version 2.0.21) using Jetpack Compose with Material 3 for the user interface, targeting Android 8.0 (API 26) as the minimum supported platform. The build system uses Gradle with the Kotlin DSL. Machine learning inference is handled by TensorFlow Lite (version 2.17.0) with optional GPU delegate support. Location data is acquired through Google Play Services FusedLocationProviderClient (version 21.0.1). Event evidence is captured using the CameraX library (version 1.3.2), and the interactive Event Map renders OpenStreetMap tiles through the osmdroid library (version 6.1.18). Trip data and user profiles are persisted locally using SharedPreferences with Gson-based JSON serialisation. Background evidence cleanup is managed through WorkManager (version 2.9.0).

The system follows a layered pipeline architecture comprising six functional layers: sensor acquisition, signal processing and feature extraction, event classification, scoring and estimation, evidence and persistence, and user interface. Data flows strictly downward through the pipeline from sensors to the UI via LiveDataBus callbacks, preventing circular module dependencies and enabling independent testability of each layer.

4.1.1 Sensor acquisition

During an active trip, the application runs as a persistent Android Foreground Service (foregroundServiceType: location|dataSync) with a visible notification, preventing the Android OS from terminating it during monitoring. The service registers listener callbacks for TYPE_LINEAR_ACCELERATION and TYPE_GYROSCOPE at SENSOR_DELAY_GAME frequency, achieving approximately 50 Hz on supported hardware. GPS speed, bearing, and coordinates are acquired through FusedLocationProviderClient at one-second intervals with PRIORITY_HIGH_ACCURACY and a 0.5-metre minimum displacement threshold.

Each accelerometer callback fuses the most recent gyroscope reading and GPS speed into a SensorSample object, which is appended to a sliding window buffer. A duplicate guard discards any sample whose values match the immediately preceding sample within a 10 ms window. When one second of elapsed time has accumulated (WINDOW_DURATION_MS = 1000), the window buffer is dispatched to the processing pipeline and cleared.

4.1.2 Signal pre-processing

Each one-second window passes through a three-stage conditioning pipeline before feature extraction. In the first stage, spike rejection discards any sample whose 3D vector magnitude exceeds 15 m/s^2, removing impulse artefacts caused by physical device impacts or sensor hardware anomalies. In the second stage, a three-point sliding median filter is applied to the forward acceleration axis to attenuate high-frequency noise while preserving step edges corresponding to genuine acceleration and braking events. In the third stage, a five-point

moving average further smooths the filtered signal to reduce residual noise before statistical feature computation.

4.2 Feature extraction

Eight statistical features are computed from each conditioned one-second window and assembled into a FeatureVector object. These are: (1) mean forward acceleration, (2) peak forward acceleration (maximum), (3) minimum forward acceleration (captures braking as a negative value), (4) standard deviation of horizontal acceleration, (5) mean gyroscope magnitude, (6) peak gyroscope magnitude, (7) gyroscope jitter (standard deviation of gyroscope magnitude), and (8) standard deviation of the vertical acceleration axis, which serves as an indicator of road roughness and instability.

These eight features were selected to capture the distinct physical signatures of the three target event types. Harsh acceleration is reflected in high mean and peak forward acceleration values. Harsh braking is reflected in a strongly negative minimum forward acceleration. Unstable riding, which encompasses weaving, road roughness response, and erratic lateral motion, is reflected in elevated horizontal standard deviation and gyroscope-derived features.

4.3 Event detection pipeline

TeleDrive provides three selectable detection modes, selectable by the rider before or during a trip through the AnalyzerProvider singleton.

4.3.1 Rule-based detection

The RuleBasedAnalyzer implements a stateless single-window classifier with speed-gated thresholds. The base thresholds are: harsh acceleration when peak forward acceleration exceeds 2.5 m/s^2 above 15 km/h; harsh braking when minimum forward acceleration is below –3.5 m/s^2 above 12 km/h; unstable riding when horizontal standard deviation exceeds 3.0 m/s^2 or mean gyroscope magnitude exceeds 2.0 rad/s above 8 km/h. These base thresholds are further modified by a set of validation layers implemented inside SensorService.processWindow: speed-adaptive threshold scaling (thresholds tighten at low speed and relax at high speed), magnitude-dominance checking (the dominant axis must exceed the competing axis by at least 20% when both features are simultaneously elevated), variance-window filtering (windows with globally low variance are rejected even if a single feature exceeds threshold), GPS speed-trend validation (harsh acceleration requires a rising speed trend across a five-window GPS buffer; harsh braking requires a falling trend), and a pattern-persistence gate requiring at least 30% of sub-windows within a mini-window analysis to corroborate the detected event type. A vibration baseline exponential moving average (alpha = 0.05) dynamically adjusts the instability threshold to current road surface conditions, requiring approximately 20 seconds to stabilise after surface changes.

4.3.2 Machine learning model

The CNN model (version 4) receives an input tensor of shape (1, 50, 9): one window, 50 time steps, and 9 feature channels per step. The nine channels are the three accelerometer axes (ax, ay, az), three gyroscope axes (gx, gy, gz), acceleration magnitude, jerk magnitude, and GPS speed. Each channel is normalised using per-feature statistics from a companion scaler.json file bundled with the APK. The model is deployed as a TFLite flat-buffer and executed through the TFLite Interpreter with optional GPU delegate. The output layer produces a four-class softmax vector corresponding to Normal, Harsh Acceleration, Harsh Braking, and Unstable Ride. The class with the highest probability is selected, and the confidence value is used as the event severity.

The model was trained on a Python workstation using TensorFlow/Keras with the following configuration: Adam optimiser with a learning rate of 0.001, sparse categorical cross-entropy loss, a maximum of 80 epochs with early stopping (patience = 10), learning rate reduction on plateau (patience = 5), and class weights applied to address residual imbalance after dataset balancing. The training dataset comprised 473,800 raw sensor samples collected across 48 real two-wheeler ride sessions. Overlapping 50-sample windows with stride 10 were extracted, and the NORMAL class was downsampled to achieve a final class-balanced dataset of 115,250 samples: 56,800 Normal (49.3%), 28,400 Harsh Acceleration (24.6%), 18,000 Harsh Braking (15.6%), and 12,050 Unstable Ride (10.5%). The original class imbalance ratio of 42:1 was reduced to 4.7:1.

4.3.3 Hybrid detection mode

In Hybrid mode (the default), both detection engines execute in parallel on the same one-second window. A consensus protocol then resolves the final classification output. When both engines produce the same event label, the output is accepted and confidence is floored at 85%, regardless of individual confidence values. When they disagree on harsh acceleration or harsh braking, the rule-based engine prevails, as these events have deterministic directional physics signatures that the threshold engine captures with higher precision. When they disagree on Unstable Ride, the ML engine's output is accepted only if its confidence exceeds 88%; below this gate, the unstable classification is suppressed. This protocol leverages the deterministic strengths of physics-based logic for directional events while relying on learned temporal patterns for the more ambiguous instability category.

4.4 Event confirmation and cooldown logic

All classification outputs — regardless of detection mode — pass through an event-confirmation state machine before any event is scored, displayed, or used to trigger evidence capture. This mechanism prevents single-window transients from generating scored events. Entry into a non-Normal state requires EVENTCONFIRMTHRESHOLD = 1 consecutive non-Normal classification. Exit back to Normal requires NORMALCONFIRMTHRESHOLD = 2 consecutive Normal classifications, implementing a hysteresis mechanism that prevents rapid state flickering and holds the event state during brief Normal interruptions within sustained manoeuvres. Two cooldown timers provide additional suppression: EVENTCOOLDOWN = 2000 ms prevents any new event from being confirmed within two seconds of the previous

event, and SCORINGCOOLDOWN = 4000 ms prevents scoring within four seconds of the previous scoring penalty. A stricter UI display threshold is also applied independently to prevent visual flicker on the live trip screen that could distract the rider.

4.5 Eco score and fuel estimation

The eco score begins at 100.0 at trip start. Each confirmed event applies a penalty computed as:

$$\text{penalty} = \text{basePenalty} \times \text{normalizedSeverity} \times \frac{1}{\sqrt{n}}$$

where basePenalty is 8 for harsh acceleration, 10 for harsh braking, and 9 for unstable riding; normalizedSeverity is the confidence or magnitude measure clamped to [0.7, 1.0]; and n is the count of prior penalties of the same type, implementing a diminishing-returns model that prevents the score from collapsing to zero during prolonged adverse riding sessions. Score colour coding follows three bands: green (90–100) for smooth riding, yellow (75–89) for moderate risk, and red (below 75) for consistently aggressive behaviour.

Fuel estimation is handled by FuelEstimator, which reduces the rider's declared base mileage by per-event degradation penalties of 0.4 km/L per harsh acceleration event, 0.3 km/L per harsh braking event, and 0.5 km/L per unstable riding event. Effective mileage is clamped between 30 and 70 km/L. Total fuel consumed equals trip distance divided by effective mileage. A parallel financial loss calculation in EcoScoreEngine tracks cumulative fuel wastage attributable to aggressive behaviour — 0.004 litres per acceleration event and 0.0025 litres per braking event — and multiplies by a configurable petrol price defaulting to ₹105 per litre.

4.6 Evidence capture and spatial analytics

When the event-confirmation state machine confirms a non-Normal event and the camera cooldown of 3 seconds has elapsed, the system launches CameraControllerActivity — an invisible 1×1 pixel CameraX activity — which captures a JPEG photograph from the rear camera using CAPTURE_MODE_MINIMIZE_LATENCY with flash disabled. Images are saved with event-type-encoded filenames and timestamps and stored in the application's private external files directory. Automatic deletion of evidence older than 24 hours is scheduled through a daily WorkManager job to limit the retention window of location-correlated imagery.

At trip completion, InsightEngine generates contextual riding insights by comparing the completed trip against historical data across seven behavioural dimensions: acceleration, braking, stability, fuel efficiency, overall improvement, streak, and milestone. The top three insights ranked by severity are included in the ride summary. TripStorage serialises the completed TripSummary — comprising eco score, distance, duration, average speed, event counts, fuel consumed, financial loss, and a list of TripEvent objects with GPS coordinates, timestamps, severity, and type — to JSON via Gson and appends it to the SharedPreferences trip history store.

The EventMapActivity renders the trip route as a polyline on an OpenStreetMap base map using osmdroid, with colour-coded markers for each confirmed event (red for harsh braking, orange for harsh acceleration, yellow for unstable riding). HotspotDetector analyses accumulated event coordinates across all trips and identifies geographic clusters of repeated unsafe behaviour using proximity grouping. Per-event coaching text and temporally matched photographic evidence are accessible through each map marker.

5. RESULTS AND DISCUSSION:

5.1 Machine learning evaluation

The CNN model was evaluated on a stratified 20% hold-out test set of 23,050 samples drawn from the balanced training dataset. Overall accuracy on this offline evaluation was 96.0% with a macro-average F1-score of 0.929. Per-class results were as follows: Normal achieved precision 0.98, recall 0.97, and F1 0.975; Harsh Acceleration achieved precision 0.93, recall 0.90, and F1 0.915; Harsh Braking achieved precision 0.95, recall 0.93, and F1 0.940; Unstable Ride achieved precision 0.90, recall 0.87, and F1 0.885. Unstable Ride exhibited the lowest per-class F1, consistent with the inherent overlap between rider-induced instability and environmental road noise in the training data.

Real-world validation on 402 labelled one-second windows from live ride sessions — processed through the complete deployed pipeline including signal conditioning and speed gating — achieved 98.76% accuracy, corresponding to only 5 misclassifications. Harsh Acceleration achieved perfect precision (1.000) and recall of 0.975 in this evaluation. The remaining misclassifications occurred between Unstable Ride and Harsh Braking, which is expected given the overlapping motion characteristics of these two classes under actual road conditions. The improvement from offline to real-world accuracy is attributable to the signal quality filtering applied during pipeline execution, which removes noisy low-confidence windows before classification.

Table 2: Per-class offline evaluation metrics — balanced hold-out test set (n = 23,050)

Class	Precision	Recall	F1-Score	Support
Normal	0.98	0.97	0.975	11,360
Harsh Acceleration	0.93	0.90	0.915	5,680
Harsh Braking	0.95	0.93	0.940	3,600
Unstable Ride	0.90	0.87	0.885	2,410
Macro Average	0.940	0.918	0.929	—
Overall Accuracy	—	—	0.960	23,050

5.2 Real-world system performance

Processing latency per one-second window — covering signal conditioning, feature extraction, classification (both engines in Hybrid mode), and state machine evaluation — remained within the one-second analysis window throughout testing, satisfying the real-time constraint. TFLite inference for the 1D-CNN completed in approximately 12 milliseconds on the test device. Memory usage remained stable during foreground monitoring sessions with no observable memory growth across extended trips. Battery consumption was sufficiently modest for practical ride recording, consistent with the offline-first design avoiding network I/O and server communication.

5.3 Eco score behaviour

Controlled scoring validation confirmed that the eco score model produces meaningful differentiation across riding behaviour profiles. Smooth rides with no confirmed aggressive events consistently retained scores of 100 or near 100. Rides involving multiple harsh acceleration and braking events produced progressively lower scores, with the diminishing-returns penalty function preventing excessive score collapse under sustained aggressive conditions. The financial loss estimator correlated with observed event counts, providing an economically interpretable translation of detected behaviour for delivery worker and commuter use cases.

5.4 Comparative analysis

TeleDrive offers several capabilities not found in combination in existing solutions accessible to individual riders. Commercial fleet telematics platforms provide high accuracy and detailed analytics but require OBD-II hardware, fleet contracts, and recurring fees, making them impractical for individual gig workers. General smartphone applications designed for cars provide some behaviour detection without additional hardware, but their detection logic is calibrated for four-wheeler dynamics and generates elevated false positives on two-wheelers. TeleDrive combines smartphone-only operation, two-wheeler-specific calibration, hybrid ML-rule detection, on-device privacy preservation, and spatial evidence-linked analytics within a single zero-subscription application.

The primary limitations of the current implementation include its dependency on a consistent phone mounting orientation (automatic orientation calibration is not yet implemented), reduced detection sensitivity at very low speeds due to speed gating, and the relative difficulty of cleanly separating rider-induced instability from road surface vibration. Validation was also conducted on a single device, single vehicle, and single geographic region, which limits the generalisability of the reported performance figures to the tested configuration.

6. CONCLUSION:

TeleDrive demonstrates that a complete two-wheeler riding behaviour telematics system is achievable using only the sensors present in a mid-range Android smartphone, without external hardware, cloud infrastructure, or subscription services. The implemented system integrates real-time sensor acquisition at 50 Hz, three-stage signal conditioning, eight-feature

statistical extraction, hybrid event detection combining physics-based rule logic with a 1D-CNN trained on 48 real ride sessions, event-confirmation state machine filtering, eco scoring with diminishing-returns penalty modelling, fuel and financial loss estimation, CameraX evidence capture, and OpenStreetMap-based spatial analytics — all operating locally within the Android Foreground Service framework on consumer hardware.

The deployed CNN model (version 4, 115,250 training samples) achieved 96.0% accuracy and a macro-average F1-score of 0.929 on offline evaluation, and 98.76% accuracy on real-world validation within the complete detection pipeline. These results confirm that on-device riding behaviour classification is practically feasible at the accuracy levels required for meaningful rider feedback. The eco score model produced consistent differentiation between smooth and aggressive riding profiles, and the financial loss indicator provided an economically relevant translation of sensor-detected behaviour into actionable awareness for the target user group of delivery riders, commuters, and students.

Future work should focus on automatic mounting orientation calibration to improve robustness across device positions, migration to a Room database for scalable trip history management, enhanced instability classification to better separate rider behaviour from environmental road noise, and multi-device validation across a wider range of smartphones, vehicle types, and geographic environments. An optional cloud synchronisation module — preserving offline-first defaults while enabling fleet-level analytics for operators — would extend the system's applicability beyond individual rider use. Within the scope of the current implementation, TeleDrive establishes a validated, privacy-conscious baseline for edge-based riding behaviour analysis on the Android platform.

REFERENCES:

[1] Abebe, E., Ayalew, B., & Bezabih, M. (2021). Smartphone-based driver behaviour monitoring using inertial measurement units: A systematic review. *IEEE Access*, *9*, 134543–134567.

[2] Handel, P., Skog, I., Wahlstrom, J., Bonawiede, F., Welch, R., Ohlsson, J., & Ohlsson, M. (2014). Insurance telematics: Opportunities and challenges with the smartphone solution. *IEEE Intelligent Transportation Systems Magazine*, *6*(4), 57–70.

[3] Ignatov, A. (2018). Real-time human activity recognition from accelerometer data using convolutional neural networks. *Applied Soft Computing*, *62*, 915–922.

[4] Johnson, D. A., & Trivedi, M. M. (2011). Driving style recognition using a smartphone as a sensor platform. In *Proceedings of the IEEE 14th International Conference on Intelligent Transportation Systems* (pp. 1609–1615). IEEE.

[5] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436–444.

[6] Ministry of Road Transport and Highways. (2022). *Road accidents in India 2022*. Government of India, Transport Research Wing.

[7] Vlakveld, W., van Nes, N., de Bruin, J., Vissers, L., & van der Kroft, M. (2019). Situation awareness is higher when drivers have had more sleep. *Accident Analysis & Prevention*, *127*, 9–

[8] World Health Organization. (2023). *Global status report on road safety 2023*. WHO Press.